

LUCRAREA 4

MICROCONTROLLERUL 8051 (I)

1. OBIECTUL LUCRĂRII

Lucrarea își propune prezentarea microcontrollerului 8051 și însușirea cunoștințelor și elementelor legate de lucrul cu acesta. Sunt înfățișate aspecte legate de arhitectura hardware:

- semnificația pinilor;
- conectarea circuitelor de memorie la magistrale;
- organizarea memoriei RAM interne;
- setul de regiștri.

De asemenea, se prezintă setul de instrucțiuni al microcontrollerului și sunt rezolvate aplicații.

2. BREVIAR TEORETIC

Un microcontroller (**microcontroller unit** - MCU) este un circuit integrat care conține în aceeași capsulă, pe lângă procesor și memorie (ROM, RAM) și circuite I/O specializate de tip porturi paralele, canale timer, căi seriale și, respectiv, circuite ce gestionează logica de întreruperi. Pot exista variante ce conțin periferice nestandardizate cum ar fi convertoare AD și, respectiv, DA, ieșiri de tip PWM (pulse width modulation), circuite watch-dog, circuite dotate cu semnale pentru magistrale I²C. Au apărut și circuite ce sunt prevăzute cu locații de memorie EEPROM (cu ștergere electrică), pentru stocarea de informații ce se mențin și după dispariția tensiunii de alimentare. Diversitatea și multitudinea de periferice incluse îl fac apt pentru controlul diverselor aplicații. Practic, cu un astfel de circuit este posibilă realizarea unui întreg microsistem de calcul. Printre firmele producătoare de microcontroloare amintim:

- | | |
|---------------------|---|
| - ZILOG | - cu familia Z-8; |
| - INTEL | - cu familia 8051 (8 biți); |
| - INTEL | - cu familia 80196 (16 biți); |
| - MOTOROLA | - cu familia 68HC11, mai nou 68HC16; |
| - MICROCHIP | - cu familiile PIC 15CXX, PIC16CXX, PIC17CXX; |
| - TEXAS INSTRUMENTS | - cu familia TMS370. |
| - NEC | - cu familia 70K |

Firma INTEL a introdus familia de microcontroloare 8051 din anul 1980. Aceasta familie este continuatoarea altor familii de microcontroloare (8035, 8048), realizată de aceeași firmă. Din 1989, firma PHILIPS (SIGNETICS) produce și ea acest tip de microcontroller, iar ulterior și alte firme au preluat producția de circuite aducându-le diverse elemente suplimentare. Astfel, firma ATMEL a introdus memoria ROM de tip FLASH (cu ștergere electrică), iar firma DALLAS a realizat circuite care pot să lucreze cu semnal de ceas de până la 30Mhz.

Microcontrolerele sunt folosite în diverse aplicații, acolo unde este nevoie de conducerea și gestionarea de procese în spații mici, de economicitate, de portabilitate, atât în mediu industrial cât și în electronica de larg consum.

Prezentarea familiei 8051

Microcontrolerul standard 8051 este realizat într-o capsulă cu 40 pini (DIL) sau 44 pini (PLCC). Pentru funcționare are nevoie de o singură sursă de tensiune (+5V). Inițial a fost realizat în tehnologie NMOS (consum $\approx 125 \div 150$ mA), iar ulterior a apărut și variante CMOS (consum $\approx 15 \div 30$ mA). Denumirea versiunii CMOS este 80C51.

Familia 8051 cuprinde diverse variante ale circuitului standard:

- circuitul 8051 conține 4Ko memorie PROM (one time programming) și 128 octeți RAM;
- circuitul 8031 conține 128 octeți RAM și nu are memorie ROM (ROMless);
- circuitul 8751 conține 4 Ko memorie EPROM și 128 octeți RAM;
- circuitul 8052 conține 8 Ko PROM și 256 octeți RAM;
- circuitul 8032 conține 256 octeți RAM și nu are memorie ROM;
- circuitul 8752 conține 8 Ko EPROM și 256 octeți RAM;
- circuitul 8754 conține 16 Ko EPROM și 256 octeți RAM;
- circuitul 8758 conține 32 Ko EPROM și 256 octeți RAM.

Toate aceste versiuni prezintă procesorul propriu-zis, 4 porturi paralele de câte 8 biți (3 având un caracter opțional), un canal pentru comunicații seriale, 2 canale timer, logică de întreruperi ce gestionează 2 întreruperi externe și 3 întreruperi interne (2 de la canalul timer și una de la canalul serial).

În familia microcontrolerului există și alte variante care prezintă următoarele noutăți:

- un canal timer suplimentar;
- ieșiri pentru generarea de impulsuri de tip PWM (pulse width modulation);
- convertor analog-digital (pe 10 biți);

Unele din aceste variante conțin 8 Ko și chiar 16 sau 32 Ko memorie ROM. Memoria RAM poate avea până la 512 octeți. Numărul pinilor este cuprins între $40 \div 68$, capsulele fiind de tipul DIL (Dual In Line), PLCC (Plastic Leaded Chip Carrier) sau CLCC (Ceramic Leaded Chip Carrier).

Pentru circuitele ce posedă memorie ROM există posibilitatea secretizării conținutului acesteia. Astfel, informația poate fi criptată, folosind o secvență de criptare aleasă de utilizator, la programarea chip-ului. Există și posibilitatea utilizării mai multor biți de siguranță în cadrul secvenței de programare, prin care se poate inhiba operația de citire a conținutului memoriei ROM din microcontroler și, respectiv, a operației de programare. Aceste opțiuni previn pirateria software.

Caracteristica microcontroloarelor din familia 8051 o constituie faptul că arhitectura procesorului este de tip HARVARD.

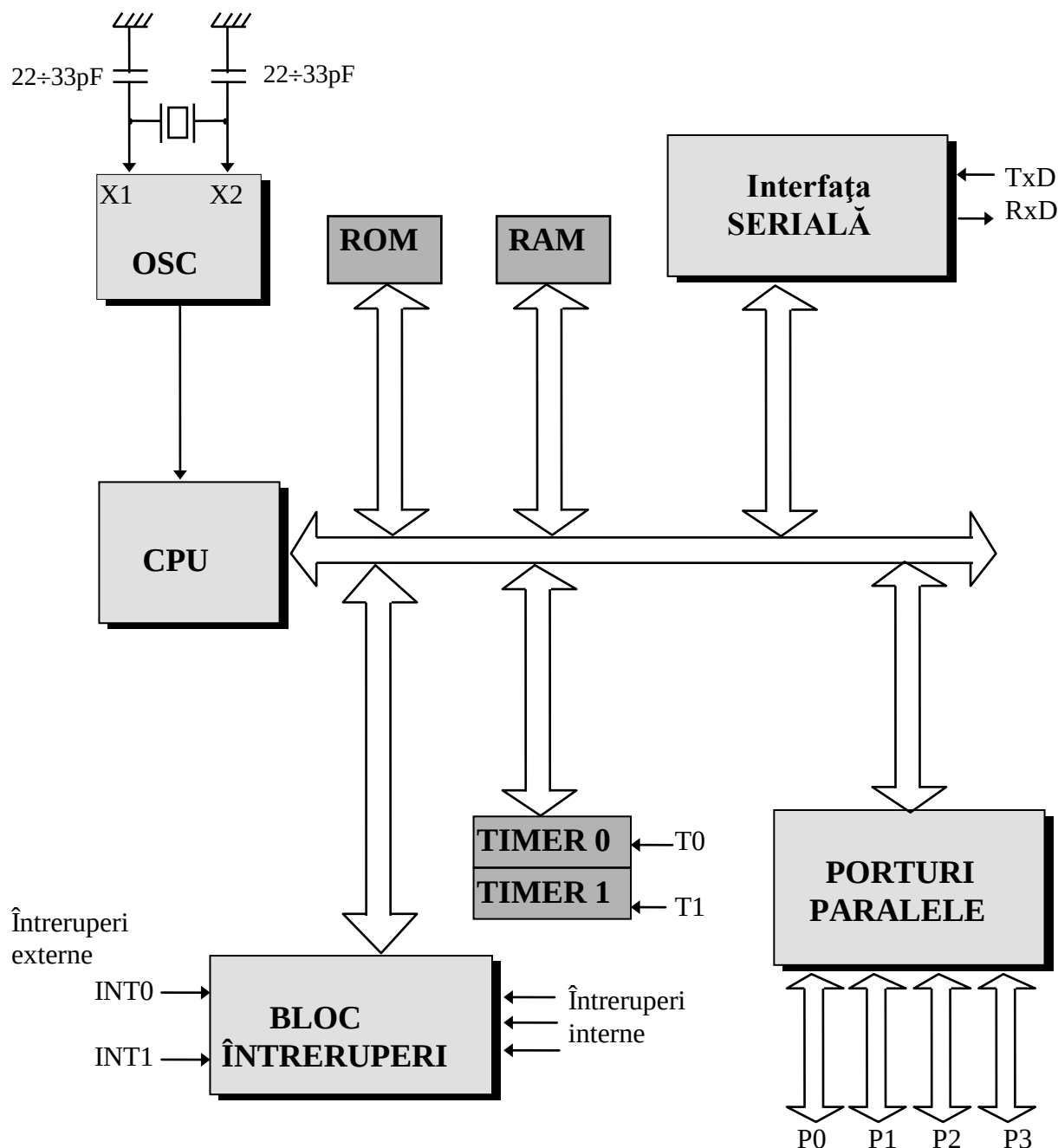
În general, procesoarele prezintă două tipuri de arhitecturi (PRINCETOWN și HARVARD).

Într-o arhitectură PRINCETOWN, instrucțiunile se plasează în același spațiu de memorie cu datele. Într-o arhitectură HARVARD, spațiul de memorie ce conține instrucțiuni diferă de spațiul de memorie ce conține date. De obicei, instrucțiunile se păstrează în memoria ROM, iar datele în RAM. Există instrucțiuni separate pentru accesarea celor două spații de memorie. Din punct de vedere hardware există semnale (pini de ieșire ai microcontrolerului) care selectează spațiul memoriei de programe separat de cel al memoriei de date. Spațiul de memorie rezervat programelor este de 64 Ko, iar cel rezervat datelor este tot de 64 Ko. Există și posibilitatea suprapunerii celor două zone de memorie cu micșorarea spațiului total de adresare la 64 Ko.

Microcontrolerele care au memorie ROM internă pot să inhide acest spațiu și să utilizeze numai memorie ROM externă.

Există și posibilitatea lucrului simultan cu memorie ROM internă și memorie ROM externă.

SCHEMA BLOC A MICROCONTROLLERULUI



Descrierea pinilor circuitului 8051

Circuitul standard realizat în capsulă DIL prezintă 40 pini. Dintre aceștia, doi pini sunt rezervați tensiunii de alimentare (+5V și masă).

Controlerul conține 4 porturi paralele de câte 8 biți fiecare, ce pot fi folosite pentru generarea și, respectiv, receptarea semnalelor digitale (notate P0, P1, P2, P3). Funcție de modul de lucru în care lucrează, aceste porturi se pot configura și pot avea mai multe situații de lucru. Astfel, dacă se lucrează cu memorie ROM externă, 2 dintre aceste porturi se constituie în linii ce formează magistralele de adrese și de date. Portul P0 generează, multiplexat în timp, partea cea mai puțin semnificativă a informației de adresă (A0-A7) și, respectiv, gestionează magistrala de date (D0-D7). Pentru formarea magistralei de adrese va fi nevoie să se plaseze în exteriorul MCU-ului un circuit de tip latch (de exemplu 74HCT573), cu scopul de a memora informația de adresă. Acționarea acestui circuit se face utilizând semnalul ALE. Portul P2, în aceleași situații, generează octetul cel mai semnificativ al informației de adrese (A8-A15). Cele 16 linii de adrese astfel formate permit adresarea a 64 Ko memorie. De fapt, se pot adresa 64Ko memorie de programe (ce conține instrucțiuni) și, respectiv, 64Ko memorie de date, deci în total 128 Ko. Notățiile liniilor porturilor sunt $P_{i,j}$, cu $i = 0, \dots, 3$, iar $j = 0, \dots, 7$.

Atunci când formează magistrala de adrese și date, liniile portului P0 mai sunt notate AD0-AD7, iar liniile lui P2 au denumirea A8-A15. Când nu se lucrează cu magistralele externe, cele 2 porturi P0 și P2 se pot folosi ca porturi de tip paralel. În această situație se atrage atenția asupra faptului că liniile portului P0 sunt de tipul "open drain", fiind necesară plasarea în exterior de rezistențe către sursa de alimentare, în vederea asigurării polarizării.

P1 este în orice situație de lucru a microcontrolerului, port de tip paralel, biții acestuia fiind configurabili atât ca ieșiri, cât și ca intrări digitale. Liniile portului sunt notate P1.0, ..., P1.7.

Ultimul port (P3) are un caracter multifuncțional, funcție de modul de lucru al microcontrolerului. Pe de o parte, prin liniile acestui port, circuitele I/O existente în MCU (cele 2 canale timer, canalul de comunicație serial, logică pentru gestionarea întreruperilor) sunt accesibile utilizatorului. Dacă nu se lucrează cu circuitele I/O locale, acest port poate să lucreze ca un port de tip paralel, liniile asigurând generarea și preluarea de semnale digitale.

Corespondența liniilor portului 3 cu liniile circuitelor I/O interne este următoarea:

- P30 - RXD - intrare pentru recepția serială a datelor la unitatea serială;
- P31 - TXD - ieșire pentru transmisia serială a datelor de la unitatea serială;
- P32 - INT0 - intrare pe care se poate aplica o cerere de întrerupere externă;
- P33 - INT1 - intrare pe care se poate aplica o cerere de întrerupere externă;
- P34 - T0 - intrare pe care se pot aplica impulsuri externe cu numărare de către timerul 0;
- P35 - T1 - intrare pe care se pot aplica impulsuri externe cu numărare de către timerul 1;
- P36 - WR - semnal pentru scriere la lucrul cu memoria de date externă;
- P37 - RD - semnal pentru citire la lucrul cu memoria de date externă;

EA - external address. Când se lucrează cu magistrale externe, acest pin se conectează la „0”-logic. Când se folosește memoria ROM internă, pinul se leagă la „1”-logic. În această situație, la adresarea ROM-ului intern de 4K, controlerul generează adrese din gama 0000h-0FFFh și nu se emit semnale pe magistralele externe (P0 și P2 putând fi folosite ca porturi paralele). Există posibilitatea folosirii în completare, a unei memorii ROM externe, dar care va fi selectată și adresată începând cu adresa 1000h (P0 și P2 încetând să mai poată fi folosite ca porturi paralele).

ALE - address latch enable. Semnalul este folosit pentru demultiplexarea externă a informațiilor emise pe portul P0. Astfel, semnalul este folosit în vederea memorării în latch-ul extern 74HCT573, a părții mai puțin semnificative a adresei A0-A7 emisă pe liniile acestuia. Semnalul este activ „1” logic.

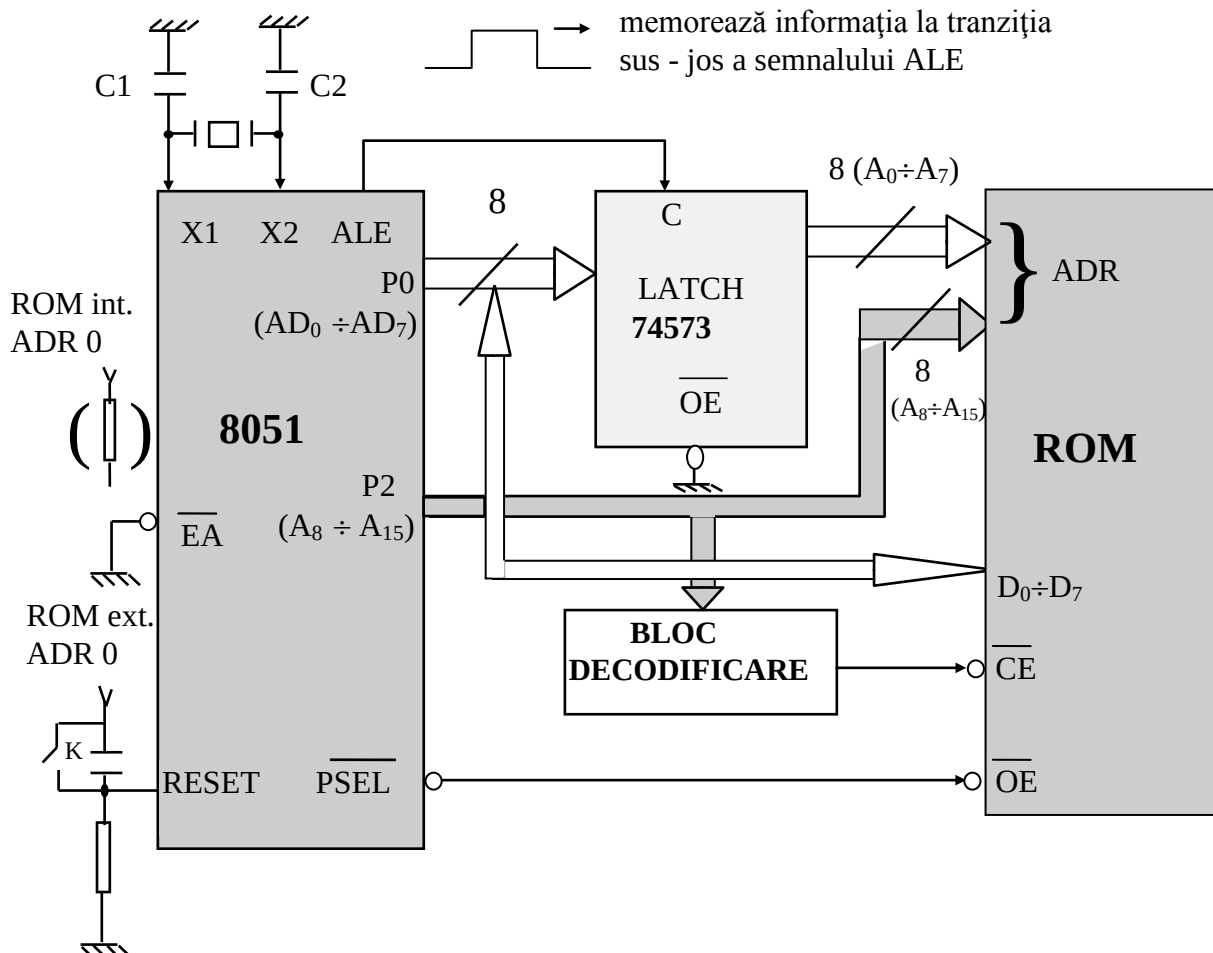
PSEL - program select. Este semnalul prin care se face activarea memoriei de programe atunci când controlerul realizează extragerea codurilor instrucțiunilor. Dacă se lucrează cu ROM-ul intern, acest semnal nu se generează. Pinul este folosit și în situațiile de programare ale ROM-ului intern. În acest caz, pinul este intrare având notați PROG (Programming).

RESET - Semnal pentru inițializarea controlerului, activ “0”- logic.

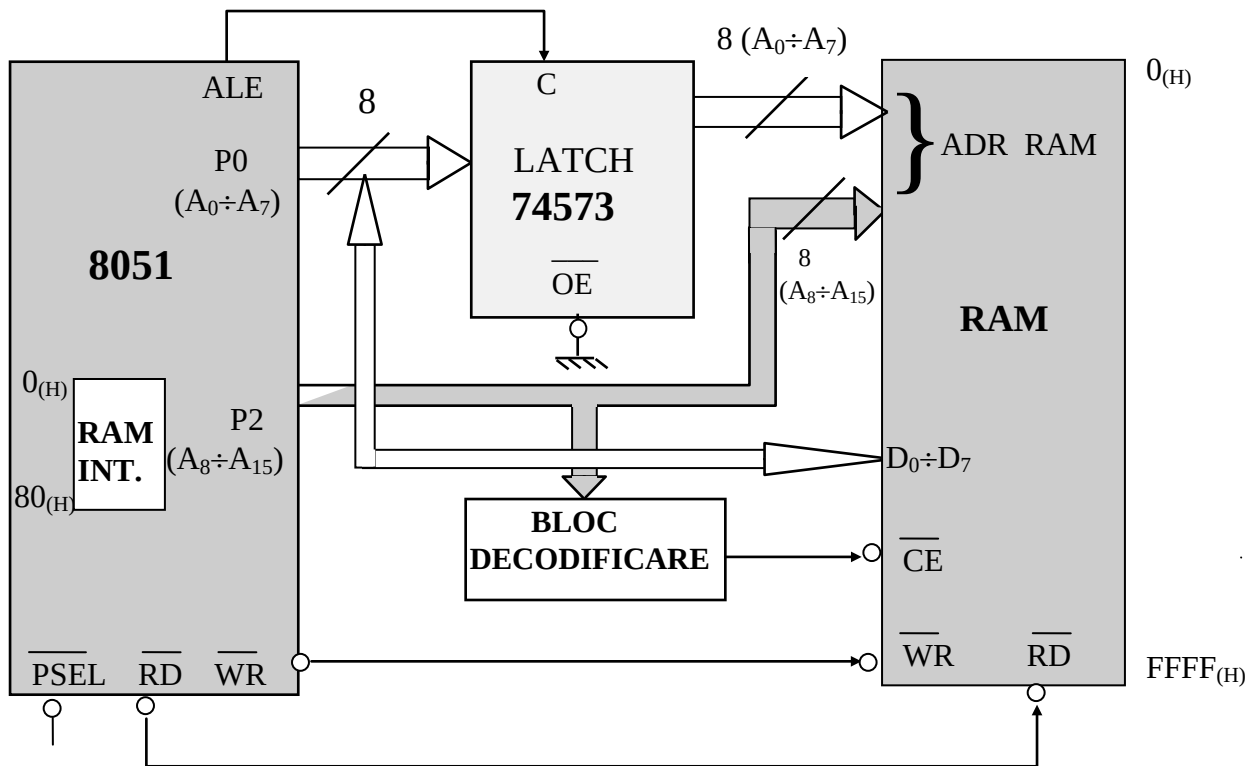
XTAL1; XTAL2 - Pini pe care se conectează în exterior cristalul de cuarț în ritmul căruia lucrează oscilatorul local. Se poate lucra și cu oscilator extern, semnalul respectiv aplicându-se pe XTAL1. În acest caz, XTAL2 amplifică semnalul în vederea folosirii de către alte circuite din respectivul microsistem care au nevoie de un astfel de semnal. Varianta standard de μC 8051 lucrează cu un cristal de cuarț de 12 MHz. Au apărut variante la 16 Mhz, ajungându-se chiar de 30 Mhz.

Modalitatea de conectare a circuitelor de memorie la magistralele controlerului 8051

1. Conectarea memoriei de programe (memoria ROM)

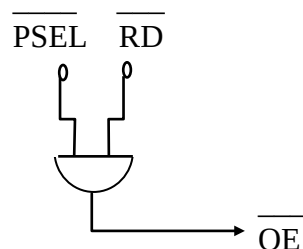


2. Conectarea memoriei de date (memoria RAM)



Intrarea $\overline{EA}$ nu are nici un efect asupra memoriei de date. Memoria RAM internă ocupă adresele $00h \div 7Fh$. RAM-ul extern se poate plasa în spațiul de adrese $0000h \div FFFFh$. Nu apare nici o interferență între cele două zone de memorie, deoarece fiecare este accesată din punct de vedere hardware prin semnale diferite, iar software se utilizează instrucțiuni specifice pentru adresare. Astfel, la adresarea memoriei de date externe se utilizează instrucțiunea MOVX, în timp ce pentru a extrage date din memoria de programe se folosește instrucțiunea MOVC.

Există și posibilitatea suprapunerii celor două spații de memorie, cu reducerea spațiului de adresare. În felul acesta, memoria RAM poate fi folosită pentru rularea programelor, dacă în prealabil în ea s-au transferat instrucțiuni. Hardware, se procedează conform schemei următoare, pentru obținerea semnalului de accesare al memoriei RAM:



Organizarea memoriei RAM interne a microcontrolerului

Memoria RAM internă este împărțită în 2 zone. O primă zonă RAM începe cu adresa 00h, fiind destinată în principal folosirii de către utilizator. Ea conține un număr de locații ce depinde de tipul respectivului controler. Dimensiunea standard este de 128 locații (octeți). Zona se adresează prin instrucțiuni specifice care nu accesează eventuala memorie RAM exterioară microcontrolerului conectată la adresele similare externe. Cele două spații de memorie RAM internă și externă sunt accesate diferit. Pentru fiecare spațiu există instrucțiuni specifice cu care se poate lucra. Când se lucrează cu spațiul de adrese interne, semnalele de adrese externe nu se activează. Zona de memorie RAM internă (la 8051, 128 de locații notate 00h - 7Fh) este și locul în care se creează stiva procesorului. Registrul SP, pe 8 biți, adresează această stivă. La depunerile în stivă, SP-ul se incrementează, iar la extragerile din stivă acesta se decrementează, stiva fiind adresată printr-o tehnică de tip LIFO (last in first out). La inițializare (reset), SP-ul se încarcă cu adresa 07h și prima depunere în stivă se va face la adresa 08h. Stiva crește către sfârșitul memoriei (invers față de microprocesorul Z80). Utilizatorul are posibilitatea să schimbe baza stivei prin încărcarea registrului SP cu o nouă adresă, printr-o instrucțiune specifică. O altă proprietate a RAM-ului local constă în faptul că locațiile cu adresele de la 20h până la 2Fh se constituie în zona adresabilă pe bit. Cu cei 128 biți ai acestei zone se poate lucra în mod individual folosind instrucțiuni specifice. În al treilea rând, în RAM-ul local se plasează și o serie de regiștri ai unității centrale (opțional 4 bank-uri de 8 regiștri generali).

Pe lângă această zonă de memorie RAM locală, există o a doua zonă de memorie RAM denumită zona regiștrilor cu funcție specială (SFR - special function registers), zonă ce cuprinde locații în care se află sediul regiștrilor ce gestionează circuitele I/O locale, precum și regiștri ai unității centrale. Din punct de vedere al adreselor, această zonă se întinde de la 80h la FFh.

F8								FF
F0	B							F7
E8								EF
E0	ACC							E7
D8								DF
D0	PSW							D7
C8								CF
C0								C7
B8	IP							BF
B0	P3							B7
A8	IE							AF
A0	P2							A7
98	SCON	SBUF						9F
90	P1							97
88	TCON	TMOD	TL0	TL1	TH0	TH1		8F
80	P0	SP	DPL	DPH			PCON	87

Un asamblor specific familiei 8051 recunoaște regiștrii interni fie prin notație, fie prin adresa ocupată în spațiul SFR.

În tabloul SFR, regiștrii aflați pe prima coloană din stânga (cei care se află la adresa 80h, 88h, 90h, 98h, A0h, A8h, ..., F0h, F8h) sunt adresabili la nivel de bit, pentru ei acționând instrucțiuni ce manevrează zona de memorie RAM locală adresabilă la nivel de bit. Biții din cadrul regiștrilor adresabili pe bit, din zona SFR, au denumiri specifice prin intermediul cărora sunt manevrați de instrucțiuni în cadrul limbajului de asamblare.

Revenind la prima zonă de memorie RAM locală, spațiul cuprins între adresele 00h și 1Fh este rezervat pentru patru seturi (bank-uri) de regiștri ai unității centrale, $R_0 \div R_7$. La un moment dat, utilizatorul are acces doar la informațiile din cadrul unui singur set de regiștri. La inițializarea procesorului este activat primul bank de regiștri, $R_0 \div R_7$, aflat în locațiile de memorie 00h - 07h. Reamintim că la inițializare, registrul SP este încărcat cu valoarea 07h. Cum în urma unei depuneri în stiva, SP-ul este incrementat, rezultă că în astfel de situații, stiva va ocupa în RAM-ul intern locații începând cu adresa 08h până la sfârșitul memoriei. Dacă se optează pentru astfel de organizare a stivei, utilizatorul nu va mai putea folosi celelalte trei seturi de regiștri de tip $R_0 \div R_7$. După cum s-a amintit, zona de memorie RAM internă cuprinsă între adresele 20h și 2Fh formează așa numita zonă de memorie RAM adresabilă pe bit ce conține $16 \times 8 = 128$ biți. Cei 128 biți sunt notați de la 00h până la 7Fh. Există instrucțiuni în setul de instrucțiuni al microcontrolerului care lucrează la nivel individual cu biții din cadrul acestei zone. În limbajul de asamblare, acești biți sunt recunoscuți fie prin notația prezentată anterior, fie prin denumirea atribuită în program. Zona de memorie RAM cuprinsă între 30h și 7Fh nu are proprietăți deosebite, putând fi utilizată de către programator pentru stocarea variabilelor pe octet.

7F _(H)								2F _(H)
30 _(H)								
7F _(H)	7E _(H)	7D _(H)	7C _(H)	7B _(H)	7A _(H)	79 _(H)	78 _(H)	
								22 _(H) 21 _(H) 20 _(H)
17 _(H)	16 _(H)	15 _(H)	14 _(H)	13 _(H)	12 _(H)	11 _(H)	10 _(H)	
0F _(H)	0E _(H)	0D _(H)	0C _(H)	0B _(H)	0A _(H)	09 _(H)	08 _(H)	
07 _(H)	06 _(H)	05 _(H)	04 _(H)	03 _(H)	02 _(H)	01 _(H)	00 _(H)	
1F _(H)								R1 R2
01 _(H)								
00 _(H)								

SETUL DE REGIȘTRI AI UNITĂȚII CENTRALE DIN MICROCONTROLLER

Registrul **ACCUMULATOR (ACC sau A)** - registru pe 8 biți ce este folosit la operațiile aritmetico-logice, pentru adresarea memoriei externe în vederea preluării de informații (atât la memoria de date cât și la cea de programe) și stocării de informații (în memoria de date). Nu se recomandă folosirea lui pentru memorări de lungă durată a informațiilor.

Registrul **B** - registru pe 8 biți ce este folosit pentru operațiile de înmulțire și împărțire.

Registrul **PSW** (Programm Status Word - cuvântul de stare program). Are structura:

D7	D6	D5	D4	D3	D2	D1	D0
CY	AC	F0	RS1	RS0	OV	—	P

Registrul conține indicatorii de condiții:

CY - indicator carry;

AC - auxiliar carry;

F0 - Flag 0 - bit ce poate fi folosit de utilizator pentru scopuri generale de semnalizare;

RS1, RS0 - register select - biți folosiți pentru selectarea bancului de regiștrii activi $R_0 \div R_7$.

Selecția se face conform tabelului:

RS1	RS0	SELECȚIE
0	0	- bancul activ în celulele $00_{(H)} \div 07_{(H)}$.
0	1	- bancul activ în celulele $08_{(H)} \div 0F_{(H)}$.
1	0	- bancul activ în celulele $10_{(H)} \div 17_{(H)}$.
1	1	- bancul activ în celulele $18_{(H)} \div 1F_{(H)}$.

OV - overflow - indicator de depășire a rezultatelor obținute la operații aritmetice efectuate asupra operanzilor exprimați în cod complement față de 2;

P - parity - indică paritatea rezultatelor obținute în acumulator.

Registrul **SP** (Stack Pointer - indicatorul vârfului stivei) - folosit pentru adresarea stivei.

Registrul **DPTR** (Data Pointer) - registru pe 16 biți alcătuit din 2 regiștri DPH și DPL. Este folosit pentru adresarea memoriei externe de date și de programe.

Registrul **PCON** (Power Control) - registru pentru gestionarea energetică a microcontrolerului.

Structura acestui registru este următoarea :

D7	D6	D5	D4	D3	D2	D1	D0
SMOD	—	—	—	GF1	GF0	PD	IDL

SMOD - (serial mode) - dacă acest bit este pus "1"- logic și dacă canalul timer 1 este folosit pentru generarea semnalului pentru ceasul transmisiei și recepției seriale, atunci rata comunicației este dublată. Dacă SMOD este "0" - logic, rata de transmisiei/recepției seriale este normală.

GF1, GF0 - (general flag 1, 0) - indicatori generali ce pot fi utilizați de programator.

PD - power down - poziționarea acestui bit în "1"- logic atrage după sine oprirea activității microcontrolerului, cu intrarea acestuia într-un regim de lucru în care puterea absorbită de la sursa de tensiune este redusă foarte mult, doar RAM-ul local menținându-se alimentat. Opțiunea

este valabilă doar pentru variantele CMOS. Din acest mod de funcționare se poate ieși doar prin inițializarea hardware a MCU-ului.

IDL - idle - dacă este "1" - logic se provoacă intrarea în modul de operare idle. În acest mod activitatea procesorului este oprită, menținându-se active circuitele locale I/O și logica pentru gestionarea întreruperilor. Această stare este una de consum redus energetic din care se poate ieși prin primirea unei întreruperi sau prin inițializare hardware. Este valabilă doar pentru versiunile realizate în tehnologie CMOS ale microcontrolerului.

SETUL DE INSTRUCȚIUNI AL MICROCONTROLLERULUI 8051

Legenda pentru notațiile folosite:

Rn - unul dintre regiștrii R0 ÷ R7 ;

direct - reprezintă adresa unei locații de memorie RAM internă microcontrolerului (număr pe 8 biți). Acest număr, 00h - 7Fh, definește chiar locația RAM căutată.

@Ri - specifică conținutul locației de memorie internă, cu adresa este specificată prin unul din regiștrii R0 și R1. Se atrage atenția ca doar regiștrii R0 și R1 posedă această proprietate.

data - specifică o data imediată care apare în formatul unei instrucțiuni. Poate fi pe 8 biți sau în cazul registrului DPTR pe 16 biți.

rel - reprezintă un deplasament (număr cu semn exprimat în cod complement față de 2 - CC2)

Operații aritmetice

1. Adunarea

a) Adunarea normală

ADD A,Rn	$A \leftarrow A + Rn$
ADD A,direct	$A \leftarrow A + (\text{direct})$
ADD A,@Ri	$A \leftarrow A + (Ri)$
ADD A,# data	$A \leftarrow A + \# \text{ data}$

b) Adunare cu transport

ADDC A,Rn	$A \leftarrow A + Rn + Cy$
ADDC A,direct	$A \leftarrow A + (\text{direct}) + Cy$
ADDC A,@Ri	$A \leftarrow A + (Ri) + Cy$
ADDC A,# data	$A \leftarrow A + \# \text{ data} + Cy$

2. Scăderea

Scădere cu împrumut - borrow

SUBB A,Rn	$A \leftarrow A - Rn - Cy$
SUBB A,direct	$A \leftarrow A - (\text{direct}) - Cy$
SUBB A,@Ri	$A \leftarrow A - (Ri) - Cy$
SUBB A,# data	$A \leftarrow A - \# \text{ data} - Cy$

3.Incrementarea_

INC A	$A \leftarrow A + 1$
INC direct	$(\text{direct}) \leftarrow (\text{direct}) + 1$
INC @Ri	$(Ri) \leftarrow (Ri) + 1$
INC Rn	$Rn \leftarrow Rn + 1$
INC DPTR	$DPTR \leftarrow DPTR + 1$

4.Decrementarea

DEC A	$A \leftarrow A - 1$
DEC direct	$(\text{direct}) \leftarrow (\text{direct}) - 1$
DEC @Ri	$(Ri) \leftarrow (Ri) - 1$
DEC Rn	$Rn \leftarrow Rn - 1$

5. Multiplicarea

MUL AB $B, A \leftarrow A \times B$

Rezultatul (ocms-ul) se obține în registrul B, și, respectiv, în A(ocmps-ul).

6. Împărțirea

DIV AB $A, B \leftarrow A / B$

Rezultatul (câtul) se obține în A, iar în B restul.

7. Ajustarea zecimală a acumulatorului

DA A

Instrucțiunea acționează doar în urma unei operații aritmetice de adunare, operanzii implicați fiind exprimați în cod BCD. În astfel de situații rezultatul operației nu apare în cod BCD. Instrucțiunea DA A reajustează rezultatul obținut transformându-l în cod BCD.

Operații logice

1. Și logic

ANL A,Rn	$A \leftarrow A \wedge Rn$
ANL A,direct	$A \leftarrow A \wedge (\text{direct})$
ANL A,@Ri	$A \leftarrow A \wedge (Ri)$
ANL A,# data	$A \leftarrow A \wedge \# \text{ data}$
ANL direct,A	$(\text{direct}) \leftarrow (\text{direct}) \wedge A$
ANL direct,# data	$(\text{direct}) \leftarrow (\text{direct}) \wedge \# \text{ data}$

2. Sau logic

ORL A,Rn $A \leftarrow A \vee Rn$
 ORL A,direct $A \leftarrow A \vee (\text{direct})$
 ORL A,@Ri $A \leftarrow A \vee (Ri)$
 ORL A,# direct $A \leftarrow A \vee \# \text{ direct}$
 ORL direct,A $(\text{direct}) \leftarrow (\text{direct}) \vee A$
 ORL direct,# data $(\text{direct}) \leftarrow (\text{direct}) \vee \# \text{ data}$

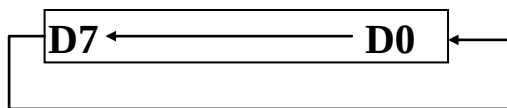
3. Sau exclusiv

XRL A,Rn $A \leftarrow A \oplus Rn$
 XRL A,direct $A \leftarrow A \oplus (\text{direct})$
 XRL A,@Ri $A \leftarrow A \oplus (Ri)$
 XRL A,# direct $A \leftarrow A \oplus \# \text{ direct}$
 XRL direct,A $(\text{direct}) \leftarrow (\text{direct}) \oplus A$
 XRL direct,# data $(\text{direct}) \leftarrow (\text{direct}) \oplus \# \text{ data}$

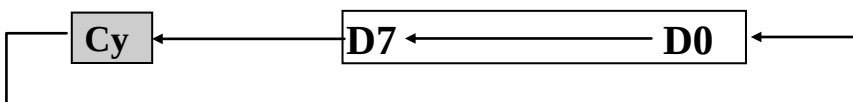
4. Alte operații logice

CLR A $A \leftarrow \underline{00}_{(H)}$
 CPL A $A \leftarrow \overline{A}$

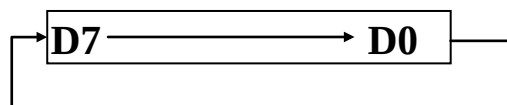
RL A - rotație a acumulatorului la stânga:



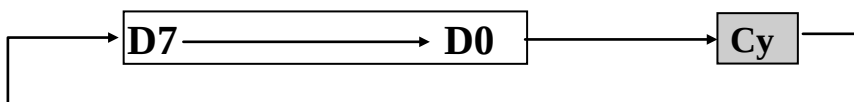
RLC A - rotație a acumulatorului la stânga prin intermediul bitului carry:



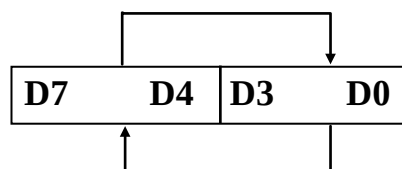
RR A - rotație a acumulatorului la dreapta:



RRC A - rotație a acumulatorului la dreapta prin intermediul bitului carry:



SWAP A - operează asupra acumulatorului conform schemei:



Operații de transfer

MOV A,Rn $A \leftarrow R_n$
 MOV A,direct $A \leftarrow (\text{direct})$
 MOV A,@Ri $A \leftarrow (R_i)$
 MOV A,# data $A \leftarrow \# \text{data}$
 MOV Rn,A $R_n \leftarrow A$
 MOV Rn,direct $R_n \leftarrow (\text{direct})$
 MOV Rn,# data $R_n \leftarrow \# \text{data}$

 MOV direct,A $(\text{direct}) \leftarrow A$
 MOV direct,Rn $(\text{direct}) \leftarrow R_n$
 MOV direct1,direct2 $(\text{direct1}) \leftarrow (\text{direct2})$
 MOV direct,@Ri $(\text{direct}) \leftarrow (R_i)$
 MOV direct,# data $(\text{direct}) \leftarrow \# \text{data}$

 MOV @Ri ,A $(R_i) \leftarrow A$
 MOV @Ri ,direct $(R_i) \leftarrow (\text{direct})$
 MOV @Ri ,# data $(R_i) \leftarrow \# \text{data}$

 MOV DPTR,# data₁₆ $DPTR \leftarrow \# \text{data}_{16}$ (pe 16 biți)

Instrucțiuni pentru transfer din memoria de programe

MOVC A,@A+DPTR $A \leftarrow (A + DPTR)$
 MOVC A,@A+PC $A \leftarrow (A + PC)$

Instrucțiuni pentru adresarea memoriei de date

MOVX A,@Ri $A \leftarrow (R_i)$ - aduce conținutul locației RAM externe adresată pe 8 biți
 MOVX A,@DPTR $A \leftarrow (DPTR)$ - aduce conținutul locației RAM externe adresată de DPTR
 MOVX @Ri,A $(R_i) \leftarrow A$
 MOVX @DPTR,A $(DPTR) \leftarrow A$

Instrucțiuni pentru lucrul cu stiva

PUSH direct $(SP + 1) \leftarrow (\text{direct})$; POP direct $(\text{direct}) \leftarrow (SP + 1)$
 $SP \leftarrow SP + 1$; $SP \leftarrow SP - 1$

Instrucțiuni de schimb

XCH A,Rn $A \leftrightarrow R_n$
 XCH A,direct $A \leftrightarrow (\text{direct})$
 XCH A,@Ri $A \leftrightarrow (R_i)$
 XCHD A,@Ri $A_{3:0} \leftrightarrow (R_i)_{3:0}$

Manipulări la nivel de bit

CLR C	$Cy \leftarrow 0$	
CLR bit	$bit \leftarrow 0$	bit reprezintă un bit din zona de memorie RAM locală
SETB C	$Cy \leftarrow 1$	adresabilă la nivel de bit. Este vorba fie de zona de memorie
SETB bit	$bit \leftarrow 1$	RAM cu adresa între 20h și 2Fh, fie un registru adresabil la
CPL C	$Cy \leftarrow \overline{Cy}$	nivel de bit din zona de memorie RAM corespunzătoare
CPL bit	$bit \leftarrow \overline{bit}$	registrelor cu funcțiuni speciale
ANL C,bit	$Cy \leftarrow Cy \wedge bit$	
ANL C,/ bit	$Cy \leftarrow Cy \wedge \overline{bit}$	
ORL C,bit	$Cy \leftarrow Cy \vee bit$	
ORL C,/ bit	$Cy \leftarrow Cy \vee \overline{bit}$	
MOV C,bit	$Cy \leftarrow bit$	
MOV C,/ bit	$Cy \leftarrow \overline{bit}$	
JC rel	$PC \leftarrow PC + rel$	- salt relativ dacă bitul Cy este '1'
JNC rel	$PC \leftarrow PC + rel$	- salt relativ dacă bitul Cy este '0'
JB bit,rel		- salt relativ dacă bitul testat este '1' - logic
JNB bit,rel		- salt relativ dacă bitul testat este '0' - logic
JBC bit,rel		- salt relativ dacă bitul testat este '1' - logic, iar bitul Cy este resetat

Instrucțiuni pentru lucrul cu subrutinele

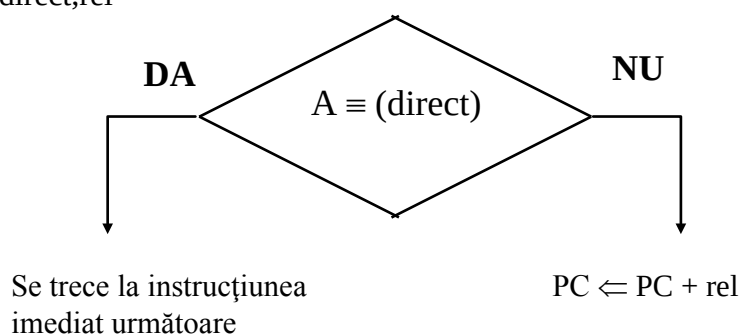
ACALL n11	- apelarea unei subrutine cu adresa precizată pe 11 biți (aflată într-un spațiu de memorie de sub 2 Ko)
LCALL n16	- apelarea unei subrutine (aflată oriunde într-un spațiu de mem. de 64 Ko)
RET	- revenire normală din subrutină
RETI	- revenire dintr-o subrutină ce tratează întreruperi

Instrucțiuni de salt

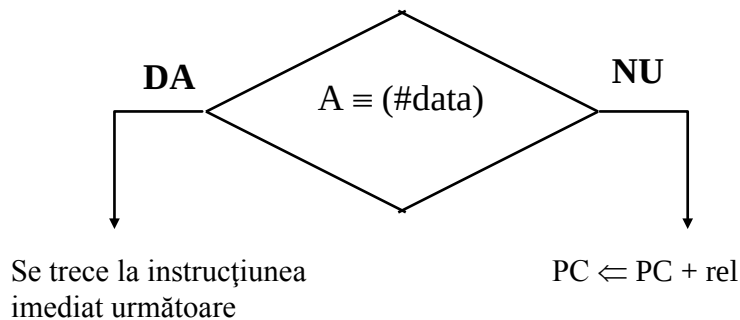
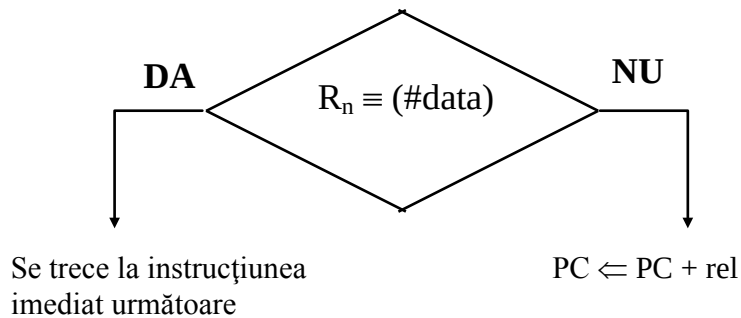
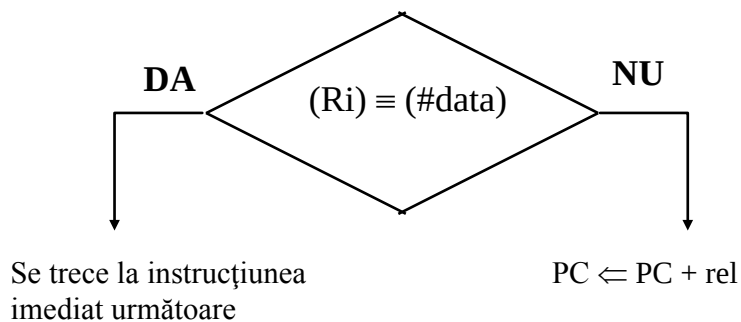
AJMP n11	$PC \leftarrow n11$	- salt necondiționat într-un spațiu de memorie de sub 2 Ko
LJMP n16	$PC \leftarrow n16$	- salt necondiționat în toată memoria de 64 Ko
SJMP rel	$PC \leftarrow PC + rel$	- salt relativ cu deplasament
JMP @A+DPTR	$PC \leftarrow A + DPTR$	
JNZ rel	$PC \leftarrow PC + rel$	- salt relativ dacă A $\neq$ 00h
JZ rel	$PC \leftarrow PC + rel$	- salt relativ dacă A = 00h

Operații de comparație

CJNE A,direct,rel



CJNE A,#data,rel

CJNE R_n,#data,relCJNE @R_i,#data,rel

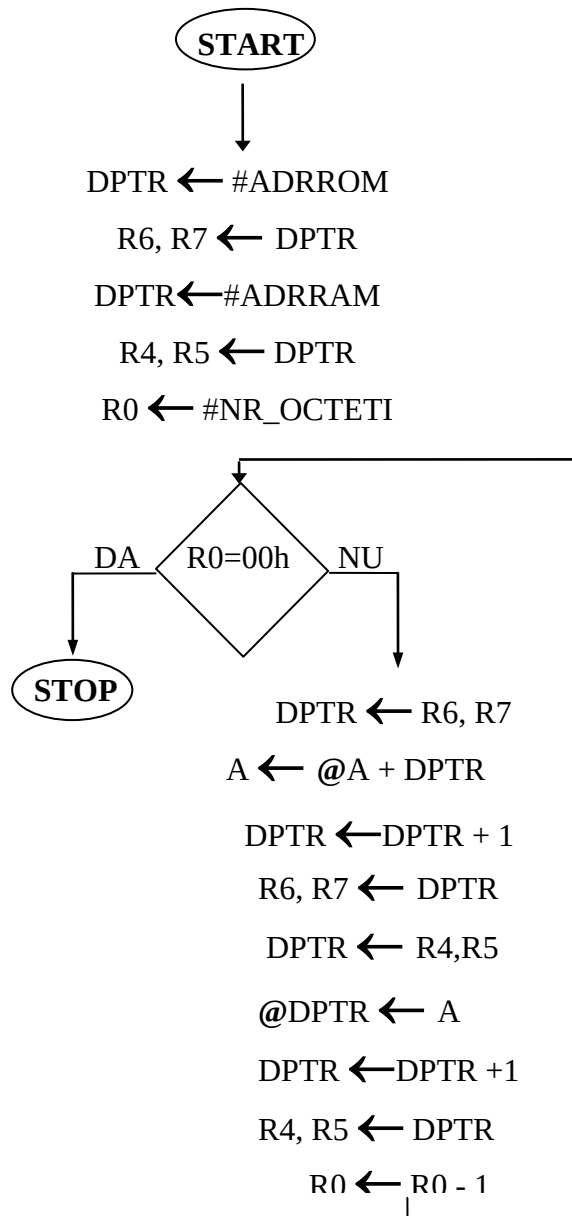
- DJNZ R_n,rel - decrementarea registru R_n și salt relativ dacă rezultatul obținut este nenul, altfel se continuă cu instrucțiunea imediat următoare
- DJNZ direct,rel - decrementarea conținutului locației de memorie RAM adresată direct și salt relativ dacă rezultatul obținut este nenul, altfel se continuă cu instrucțiunea imediat următoare
- NOP - (no operation) - nici o operație

3. PROBLEME REZOLVATE

1. Un MCU 8051 prezintă în spațiul memoriei de programe, la adresa 100h, un șir de 100 octeți, ce trebuie translatat în spațiul memoriei de date (RAM), de la adresa 2000h. Realizați un program ce permite transferul celor 100 de octeți succesivi din memoria ROM în memoria RAM.

Soluție

Pentru realizarea transferurilor cerute este necesară folosirea registrului DPTR, atât la adresarea memoriei de programe, cât și la adresarea memoriei de date. Pentru ca acest lucru să fie posibil este nevoie de salvarea adreselor termenilor din cele două șiruri de date. Vom folosi în acest sens regiștrii R6 și R7 pentru salvarea adresei sursei (a termenilor din memoria ROM), respectiv, regiștrii R4 și R5 pentru salvarea adresei destinație (a termenilor din memoria RAM). Registrul DPTR se salvează prin cele 2 componente ale sale DPH și DPL. Registrul R0 va fi folosit pe post de contor al numărului de transferuri care se efectuează. Organigrama și, respectiv, forma sursă a programului în limbaj de asamblare sunt prezentate în continuare:



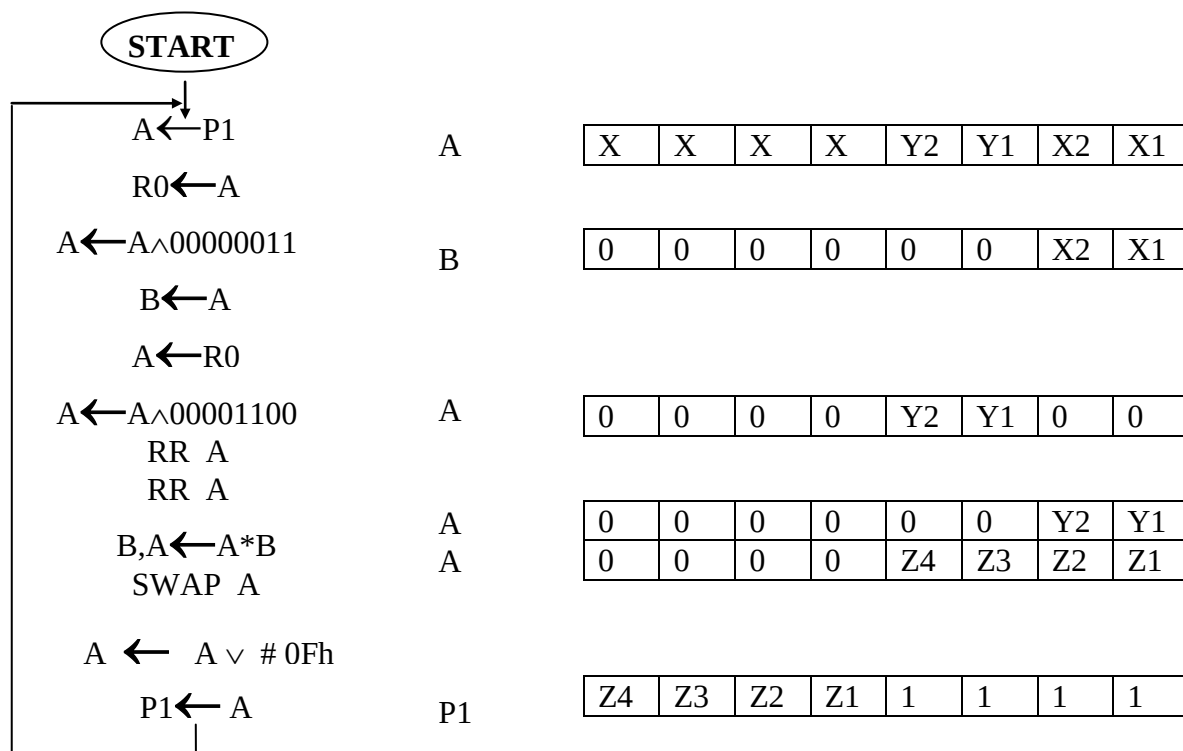
```

ORG 0h
ADRROM EQU 100h
ADRRAM EQU 2000h
NR_OCTETI EQU 100
START: MOV DPTR, #ADRROM
      MOV R6, DPH
      MOV R7, DPL
      MOV DPTR, #ADRRAM
      MOV R4, DPH
      MOV R5, DPL
      MOV R0, #NR_OCTETI
ett1: CJNE R0, #00h, ett2
      SJMP ett3
ett2: MOV DPH, R6
      MOV DPL, R7
      MOV A, #00h
      MOVC A, @A+DPTR
      INC DPTR
      MOV R6, DPH
      MOV R7, DPL
      MOV DPH, R4
      MOV DPL, R5
      MOVX @DPTR, A
      INC DPTR
      MOV R4, DPH
      MOV R5, DPL
      DEC R0
      SJMP ett1
ett3: SJMP ett3
      .END
  
```


2. Un MCU 8051 primește pe liniile P1.0 și P1.1 două semnale digitale TTL, ce formează un număr binar pe 2 biți. Același 8051 primește pe liniile P1.2 și P1.3 alte două semnale TTL ce formează un al doilea număr binar pe doi biți. Realizați programul ce oferă în mod permanent, pe liniile P1.4, P1.5, P1.6 și P1.7 semnalele TTL ce reprezintă produsul aritmetic al primelor două numere. Pentru cei doi operanzi, P1.0 și, respectiv, P1.2 reprezintă biții cei mai puțini semnificativi, iar pentru produs P1.4 este bitul cel mai puțin semnificativ.

Soluție

Rezolvarea problemei presupune preluarea operanzilor și izolarea acestora. Astfel, primul operand, izolat după o operație de mascare, este transferat în registrul B. Cel de-al doilea operand este izolat în registrul A, după ce a suferit o operație de mascare logică și două rotații simple la dreapta. Rezultatul înmulțirii se obține tot în acumulator. În vederea generării pe liniile portului P1 se realizează o operație de rotație la nivel de nibble. În final, după o operație de mascare logică se obține octetul ce va fi transmis portului P1. Ultima mascare este necesară pentru a nu afecta operanzii aplicați din exterior pe liniile P1.0 - P1.3. Mai jos sunt prezentate organigrama și, respectiv, programul sursă. Atașat organigramei se prezintă și evoluția operanzilor și a rezultatului în urma prelucrărilor efectuate.



```

START:  ORG 0h
        MOV R0, A
        ANL A, #03h
        MOV B, A
        MOV A, R0
        ANL A, #0Ch
        RR A

```

```

        RR A
        MUL AB
        SWAP A
        ORL A, #0Fh
        MOV P1, A
        JMP START
        END

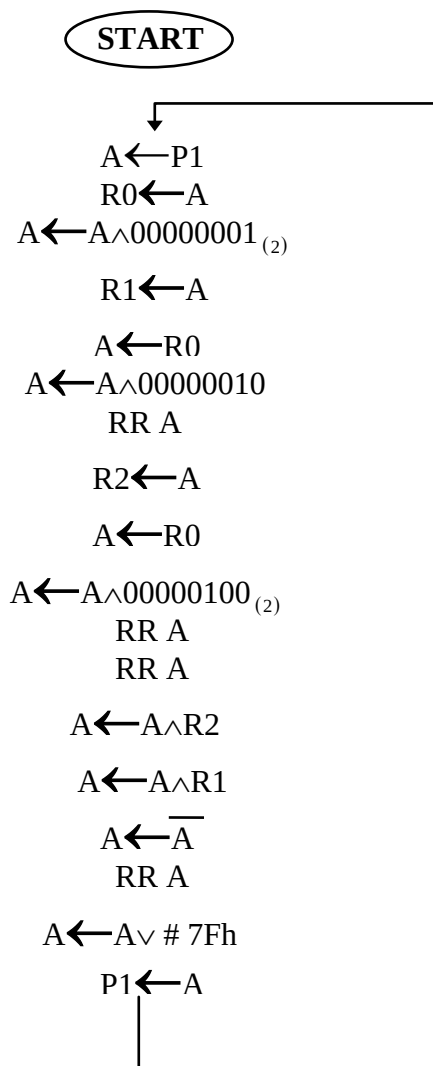
```

3. Un MCU 8051 primește din exterior pe liniile P1.0, P1.1 și P1.2, trei semnale digitale. Realizați un program prin intermediul căruia pe linia P1.7 se obține un semnal ce respectă permanent următoarea ecuație logică:

$$P1.7 = \overline{P1.0 \wedge P1.1 \wedge P1.2}$$

Soluție

Propunem o soluție care utilizează instrucțiuni logice ce operează pe octet. Operanzii sunt preluați și suferă mascări logice. Prin operații de rotație dreapta, succesive, se obține forma finală a acestora sub care va avea loc procesarea. Primii doi operanzi sunt salvați în regiștrii R1 și R2, iar ultimul în acumulator. Rezultatul operației logice suferă o mascare logică, după care este emis pe portul P1. Mascarea este necesară pentru a nu afecta operanzii aplicați la intrări. În continuare, se prezintă organigrama și forma sursă a programului în limbaj de asamblare:



```

START:  ORG 0h
        MOV A,P1
        MOV R0,AA
        ANL A,#01h
        MOV R1,A
        MOV A,R0
        ANL A,#02h
        RR A
        MOV R2,A
        MOV A,R0
        ANL A,#04h
        RR A
        RR A
        ANL A,R2
        ANL A,R1
        CPL A
        RR A
        ORL A,#7Fh
        MOV P1,A
        JMP START
        .END
  
```

4. DESFĂȘURAREA LUCRĂRII

1. Se va citi și conspecta breviarul teoretic. Se atrage atenția asupra faptului că noțiunile căpătate în lucrare vor fi folosite și la derularea lucrărilor următoare.

2. Se vor studia problemele rezolvate, urmărind și alte posibilități de rezolvare.

3. Se vor rezolva următoarele probleme propuse:

a). În memoria de programe a unui microcontroler 8051 se află plasat un șir succesiv de date ce conține 50 de octeți, începând cu adresa 2000h. Se cere transferul acestor date în memoria RAM locală a microcontrolerului, în locații succesive începând cu adresa 70h, în sens descrescător.

b). În memoria de programe a unui microcontroler 8051 se află plasat un șir de 80 date succesive, diferite, începând cu adresa 3000h. Se cerea aflarea maximului acestora și plasarea lui la adresa de RAM intern 40h.

c). Pe liniile P1.0, P1.1, P1.2 și P1.3 ale unui microcontroler 8051, se aplică semnale digitale TTL. Se cere realizarea unui program prin intermediul căruia pe linia P1.7 se obține în mod permanent un semnal ce reprezintă funcția logică SAU-EXCLUSIV a celor 4 semnale de intrare.

d). Două numere reprezentate pe câte 2 octeți fiecare sunt plasate în memoria RAM locală a unui microcontroler 8051 la adresele 40h și 50h începând cu o.c.m.p.s. Se cere obținerea produsului celor două numere și plasarea rezultatului în memoria RAM locală începând cu adresa 60h și cu octetul cel mai pu.s.

4. Se va răspunde la următoarele întrebări:

a). Pentru situația primei probleme, este posibilă salvarea registrului DPTR în stiva, pentru cele două situații de adresare a memoriei (de date și de programe)? Justificați.

b). Pentru situația primei probleme rezolvate, indicați o altă posibilitate de salvare a conținutului regiștrilor DPH și DPL, diferită de cea prezentată în rezolvare.

c). În cazul celei de-a doua probleme rezolvate, pentru izolarea celui de-al doilea operand se putea folosi operația RRC A, în locul operației RR A ? Justificați.

d). Cum putea fi înlocuită operația SWAP A în soluția prezentată la rezolvarea celei de-a doua probleme ?

e). Instrucțiunea MUL AB afectează și registrul B al microcontrolerului 8051. De ce rezultatul obținut în acest registru nu este folosit în rezolvarea problemei a doua ? În urma operației de înmulțire, care este rezultatul obținut în registrul B ?

f). Pentru cazul celei de-a treia probleme rezolvate dați o explicație legată de mascarea finală (ce folosește instrucțiunea ORL). De ce credeți că este nevoie ca biții utilizați ca intrări să fie poziționați la emisie de către microcontroler, în starea "1"- logic ?

g). Indicați metoda de rezolvare a celei de-a treia probleme care să utilizeze operații logice pe bit în locul celor pe octet prezentate în rezolvare.

h). Prezentați o metodă prin care se poate realiza o temporizare software în limbajul de asamblare a controlerului 8051.